

🚩 Flag 0 — Create Your Feature Branch

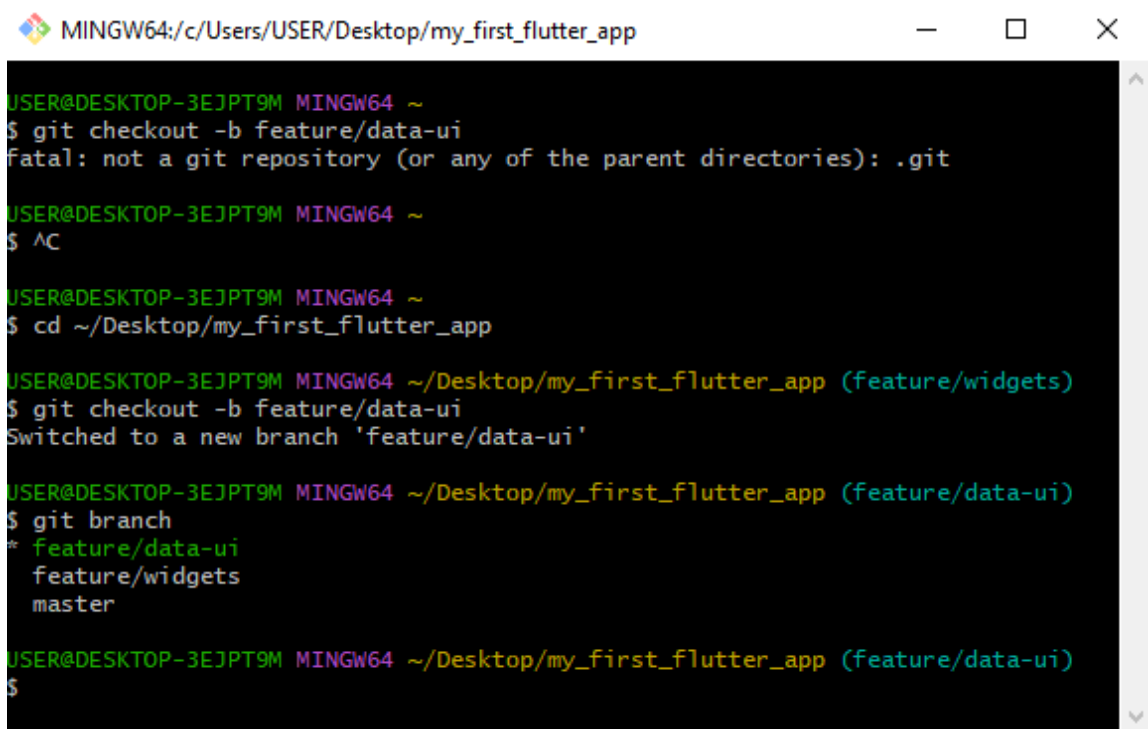
Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch



```
MINGW64:/c/Users/USER/Desktop/my_first_flutter_app

USER@DESKTOP-3EJPT9M MINGW64 ~
$ git checkout -b feature/data-ui
fatal: not a git repository (or any of the parent directories): .git

USER@DESKTOP-3EJPT9M MINGW64 ~
$ ^C

USER@DESKTOP-3EJPT9M MINGW64 ~
$ cd ~/Desktop/my_first_flutter_app

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/widgets)
$ git checkout -b feature/data-ui
Switched to a new branch 'feature/data-ui'

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$ git branch
* feature/data-ui
  feature/widgets
  master

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$
```

🚩 Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Create:

- Your identified data
- The corresponding data types

Identified Data	Data Types
Name	String
Course	String
Age	Int
Hobby	String
Height	Double
Student Status	Bool

Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

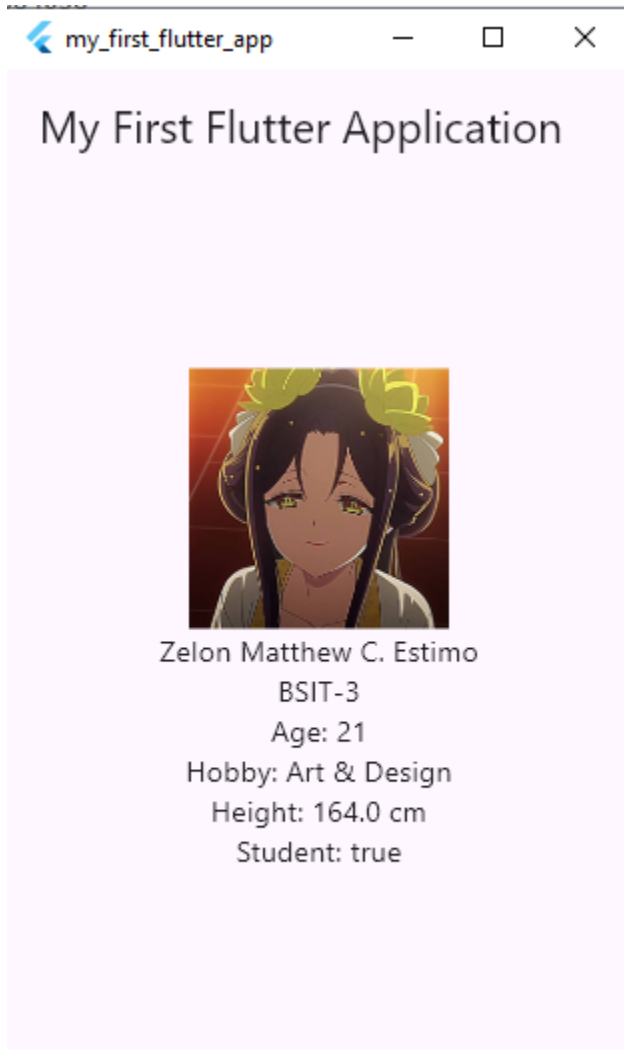
Create variables for at least:

- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**. (Declare variables separately from your UI, not directly inside your widgets)

Screenshot:

- Your variables
- Their data types
- Application still running



Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables

```
// Variables
String profileImage = "assets/kourerin.png";
String appTitle = "My First Flutter Application";
String name = "Zelon Matthew C. Estimo";
String courseSection = "BSIT-3";
String birthdate = "June 11, 2005";
int age = 21;
String hobby = "Art & Design";
double height = 164.0;
bool isStudent = true;
```

- Widgets displaying the variables

```
Image.asset(profileImage, width: 130),
SizedBox(height: 10),

Text(
  appTitle,
  style: TextStyle(
    fontSize: 23,
    fontWeight: FontWeight.bold,
  ), // TextStyle
), // Text

SizedBox(height: 50),

Text(
  name,
  style: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
  ), // TextStyle
), // Text

SizedBox(height: 10),

Text(
  courseSection,
  style: TextStyle(fontSize: 18),
), // Text
```

```
    SizedBox(height: 10),

    Text(
      birthdate,
      style: TextStyle(fontSize: 18),
    ), // Text

    SizedBox(height: 10),

    Text(
      "Age: ${age.toString()}",
      style: TextStyle(fontSize: 18),
    ), // Text

    SizedBox(height: 10),

    Text(
      "Hobby: $hobby",
      style: TextStyle(fontSize: 18),
    ), // Text

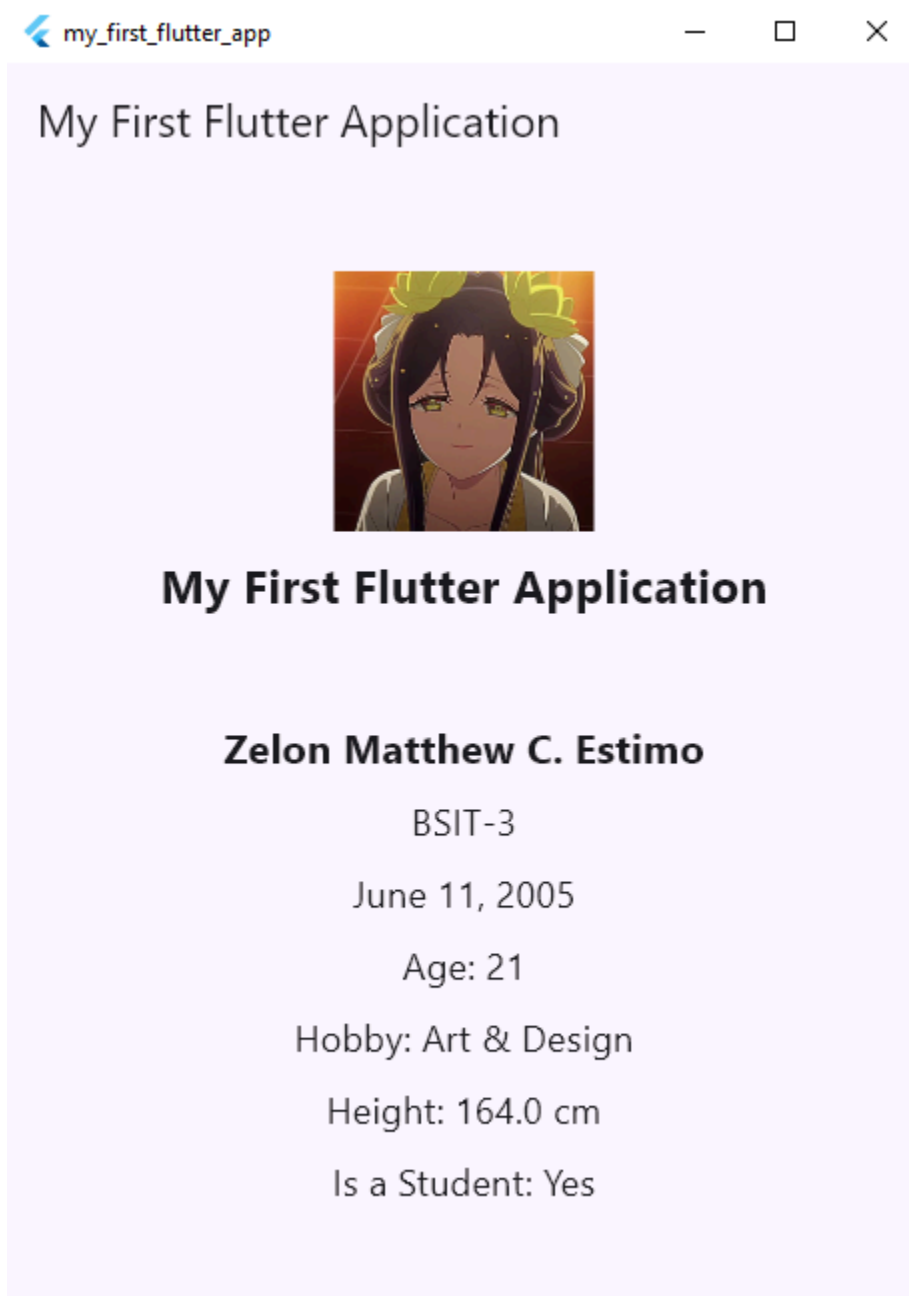
    SizedBox(height: 10),

    Text(
      "Height: $height cm",
      style: TextStyle(fontSize: 18),
    ), // Text
```

```
SizedBox(height: 10),

Text(
  isStudent ? "Is a Student: Yes" : "Is a Student: No",
  style: TextStyle(fontSize: 18),
), // Text
```

- Updated application



Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

- Image variable

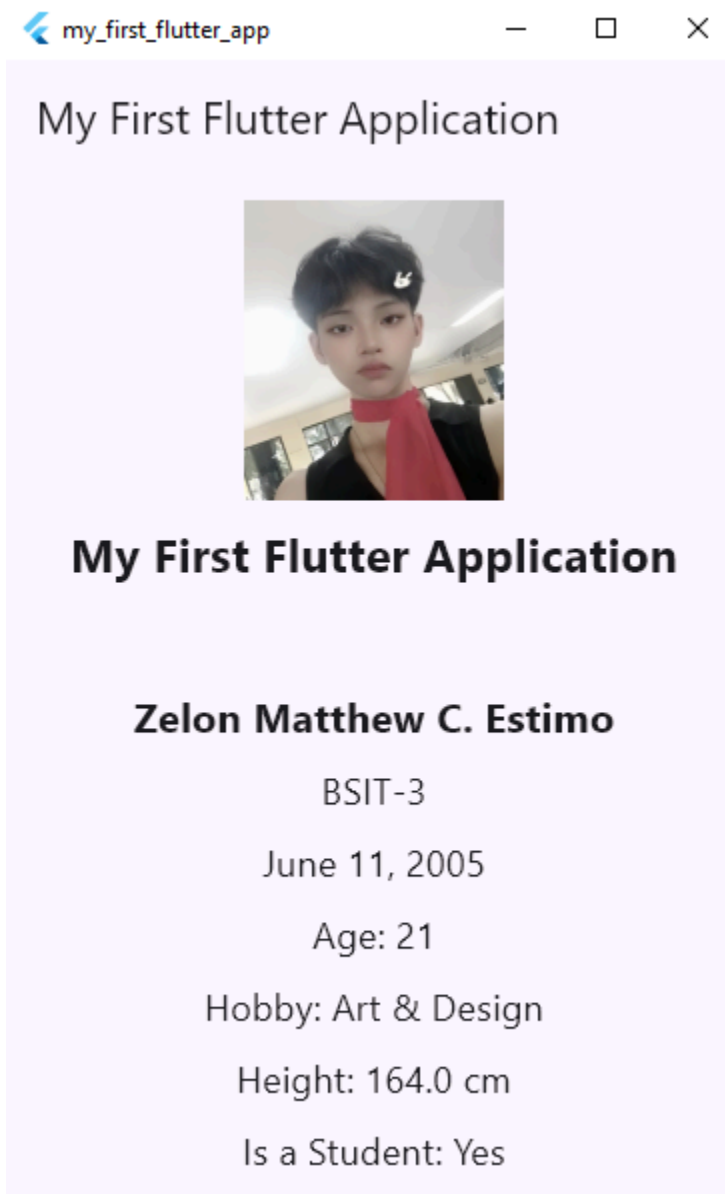
```
// Variables
String profileImage = "assets/kourerin.png";
String appTitle = "My First Flutter Application";
String name = "Zelon Matthew C. Estimo";
String courseSection = "BSIT-3";
String birthdate = "June 11, 2005";
int age = 21;
String hobby = "Art & Design";
double height = 164.0;
bool isStudent = true;
```

```
// Variables
String profileImage = "assets/zel.png";
String appTitle = "My First Flutter Application";
String name = "Zelon Matthew C. Estimo";
String courseSection = "BSIT-3";
String birthdate = "June 11, 2005";
int age = 21;
String hobby = "Art & Design";
double height = 164.0;
bool isStudent = true;
```

- Image widget using the variable

```
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Image.asset(profileImage, width: 130),
```

- Updated application



Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data

```
import 'package:flutter/material.dart';

// Profile class
class Profile {
  String image;
  String name;
  String courseSection;
  int age;
  String hobby;

  Profile({
    required this.image,
    required this.name,
    required this.courseSection,
    required this.age,
    required this.hobby,
  });
}
```

```
// Five different profiles
List<Profile> profiles = [
    Profile(
        image: 'assets/zelon.png',
        name: 'Zelon Estimo',
        courseSection: 'BSIT-3',
        age: 21,
        hobby: 'Photographer',
    ),

    Profile(
        image: 'assets/ryan.png',
        name: 'Ryan Porquiado.',
        courseSection: 'BSIT-4',
        age: 21,
        hobby: 'Drilling',
    ),

    Profile(
        image: 'assets/leon.png',
        name: 'Leonardo Cajés Jr.',
        courseSection: 'BSTM-4',
        age: 21,
        hobby: 'Twerking',
    ),

    Profile(
        image: 'assets/jackie.png',
        name: 'Jacklyn Reyes',
        courseSection: 'BSTM-4',
        age: 20,
        hobby: 'Fishing',
    ),

    Profile(
        image: 'assets/julie.png',
        name: 'Julie Ann Malinao',
        courseSection: 'BSIT-4',
        age: 21,
        hobby: 'Dogshowing Self',
    ),
];
```



Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

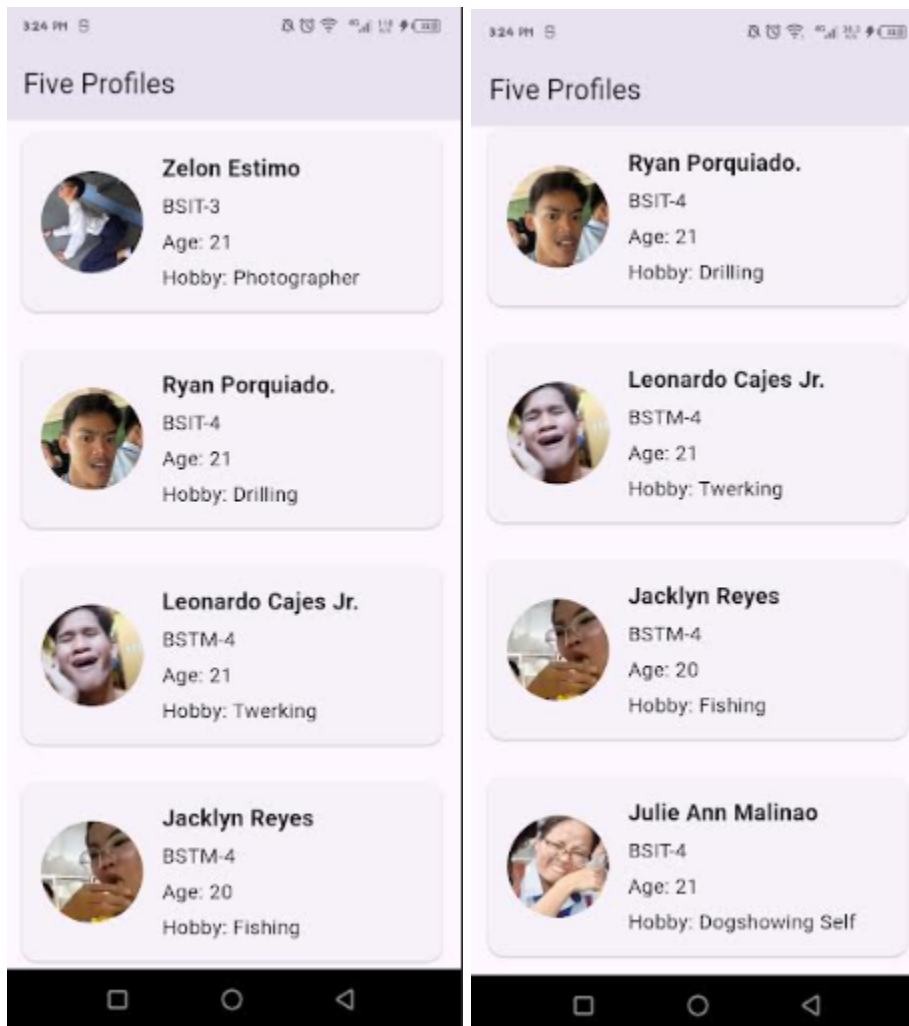
Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

- The scrolling widget in your code
- Top portion of the application

- The rest of the portion of the application showing the later profiles



🚩 Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: **missing**

Profile B

Name: Juan

Course: **missing**

Hobby: Gaming

Profile C

Name: **missing**

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

```
List<Profile> profiles = [
    Profile(
        image: 'assets/zelon.png',
        name: 'Zelon Estimo',
        courseSection: null,
        age: 21,
        hobby: 'Photographer',
    ),

    Profile(
        image: 'assets/ryan.png',
        name: null,
        courseSection: 'BSIT-4',
        age: 21,
        hobby: 'Drilling',
    ),

    Profile(
        image: 'assets/leon.png',
        name: 'Leonardo Cajés Jr.',
        courseSection: null,
        age: 21,
        hobby: 'Twerking',
    ),

    Profile(
        image: 'assets/jackie.png',
        name: 'Jacklyn Reyes',
        courseSection: 'BSTM-4',
        age: null,
        hobby: 'Fishing',
    ),
]
```

```
import package:flutter/material.dart ;

// Profile class
class Profile {
  String? image;
  String? name;
  String? courseSection;
  int? age;
  String? hobby;

  Profile({
    required this.image,
    required this.name,
    required this.courseSection,
    required this.age,
    required this.hobby,
  });
}
```

```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(
      profile.name ?? 'Unknown',
      style: const TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.bold,
      ), // TextStyle
    ), // Text

    const SizedBox(height: 5),

    Text(
      profile.courseSection ?? 'Missing',
      style: const TextStyle(fontSize: 16),
    ), // Text

    const SizedBox(height: 5),

    Text(
      "Age: ${profile.age ?? 'Missing'}",
      style: const TextStyle(fontSize: 16),
    ), // Text

    const SizedBox(height: 5),
  ],
)

```

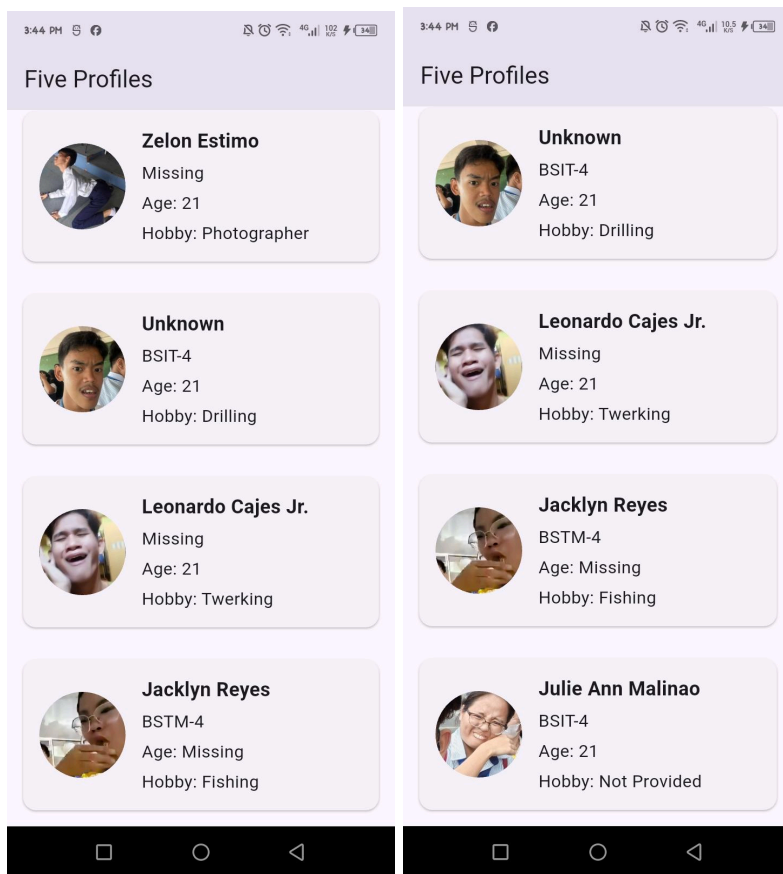
```

Text(
  "Hobby: ${profile.hobby ?? 'Not Provided'}",
  style: const TextStyle(fontSize: 16),
), // Text
],

```

- Incomplete profile data
- Application displaying fallback values

- Application still functioning correctly



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit/Terminal showing the commit

```
MINGW64:/c/Users/USER/Desktop/my_first_flutter_app

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$ git commit -m "feat: complete data-driven UI"
[feature/data-ui 9086992] feat: complete data-driven UI
 9 files changed, 111 insertions(+), 175 deletions(-)
 delete mode 100644 my_first_flutter_app/assets/Zel.png
 create mode 100644 my_first_flutter_app/assets/jackie.png
 create mode 100644 my_first_flutter_app/assets/julie.png
 delete mode 100644 my_first_flutter_app/assets/kourerin.png
 create mode 100644 my_first_flutter_app/assets/leon.png
 create mode 100644 my_first_flutter_app/assets/ryan.png
 create mode 100644 my_first_flutter_app/assets/zelon.png

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$ git push -u origin feature/data-ui
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 4 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (12/12), 1.80 MiB | 1.20 MiB/s, done.
Total 12 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/data-ui' on GitHub by visiting:
```

- Current branch

```
MINGW64:/c/Users/USER/Desktop/my_first_flutter_app

I/gralloc( 6544): unregister: id=27700002a29
Lost connection to device.

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$ git branch
* feature/data-ui
  feature/widgets
  master

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$ git checkout feature/data-ui
D      Estimo_IT314AB_Capture The Flag 2.pdf
D      my_first_flutter_app/assets/Zel.png
D      my_first_flutter_app/assets/kourerin.png
M      my_first_flutter_app/lib/main.dart
M      my_first_flutter_app/pubspec.yaml
Already on 'feature/data-ui'

USER@DESKTOP-3EJPT9M MINGW64 ~/Desktop/my_first_flutter_app (feature/data-ui)
$ git status
On branch feature/data-ui
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
```